

Explanation

Pawan Mali

June 2026

1 Provided Repository structure Explanation

I analyzed the local folder. It contains two ROS 2 workspaces:

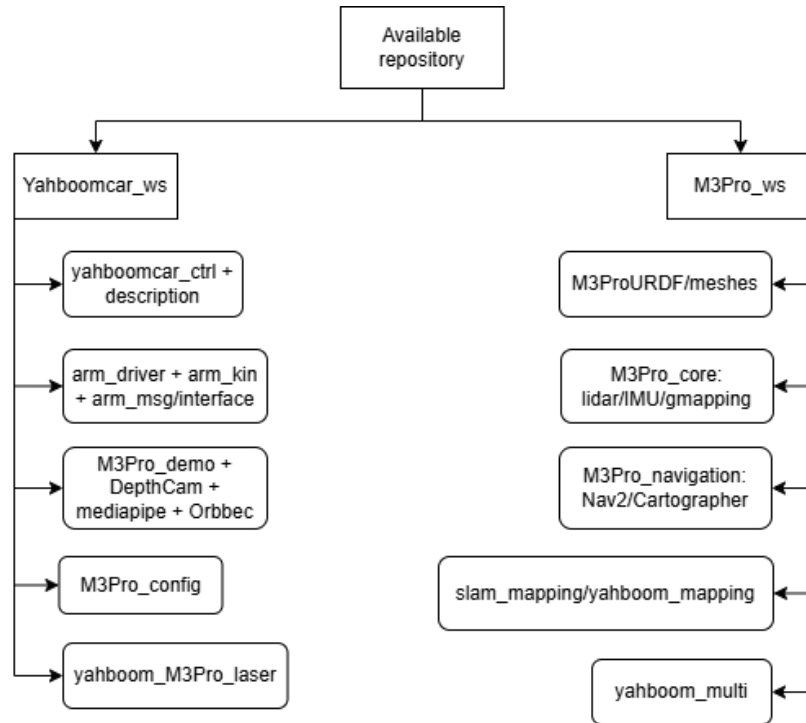


Figure 1: Overview of the downloaded repository structure

1.1 yahboomcar_ws/src

- **yahboomcar_ctrl**: Keyboard and joystick teleoperation. Important files include `yahboom_keyboard.py` and `yahboom_joy_M3Pro.py`.
- **yahboom_M3Pro_description**: Contains the robot URDF, meshes, and RViz model display files.
- **M3Pro_config**: MoveIt2 configuration package containing arm planning settings and fake `ros2_control` configuration.
- **M3Pro_demo**: Includes computer vision and manipulation demonstrations such as AprilTag detection, color tracking, KCF tracking, grasping, and line-following.
- **yahboom_M3Pro_laser**: Lidar-based applications including obstacle avoidance, obstacle warning, and target tracking.
- **yahboom_M3Pro_DepthCam**: Depth camera examples including distance measurement, depth-color visualization, and edge detection.
- **OrbbecSDK_ROS2**: ROS 2 driver package for Orbbec/Dabai depth cameras.
- **arm_interface**, **arm_msgs**: Custom messages and services used for robotic arm communication.
- **arm_kin**: Forward and inverse kinematics services implemented using KDL.
- **arm_driver**: Serial communication driver for the robotic arm using the Rosmaster library.

1.2 M3Pro_ws/src

- **M3Pro**: Robot URDF, meshes, and model display launch files.
- **M3Pro_core**: Core robot packages including laser merging, laser filtering, IMU filtering, and GMapping support.
- **M3Pro_navigation**: Navigation stack configuration including Nav2, Cartographer localization, and RViz navigation setups.
- **slam_mapping**, **yahboom_mapping**: Packages used for SLAM and map generation/saving.
- **ekf_bringup**: Robot localization package that fuses `/odom_raw` and `/imu/data` using an Extended Kalman Filter (EKF).
- **calibration**: Linear and angular calibration utilities based on `/cmd_vel` and TF data.

- **yahboom_multi**: Multi-robot navigation and coordination configurations.
- **patrol**: Laser-based autonomous patrol behavior implementation.
- **largemodel**, **largemodel_arm**: AI, voice interaction, and computer vision application examples.

2 Complete Hardware and software based pipeline

2.1 Hardware pipeline

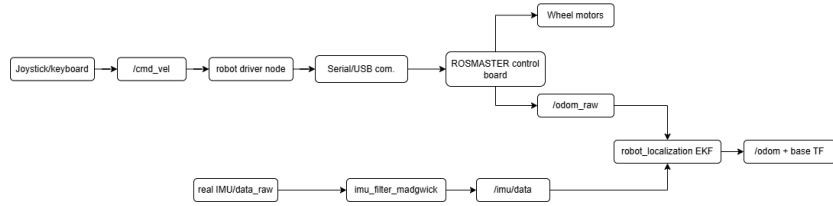


Figure 2: Hardware control pipeline overview

In the hardware-based system, movement commands are generated by a keyboard, joystick, and published on the `/cmd_vel` topic. These commands are sent through the robot driver and serial communication interface to the ROSMASTER control board, which controls the motor driver and wheel motors to move the robot. Wheel encoders provide raw odometry data, while the IMU publishes motion data. The IMU data is processed using the Madgwick filter to obtain a cleaner orientation estimate. Finally, the robotlocalization EKF uses the odometry and IMU data to produce accurate robot pose information (`/odom`) transformation, which are used by RViz2, SLAM, and Nav2 for localization and navigation.

2.2 Software pipeline

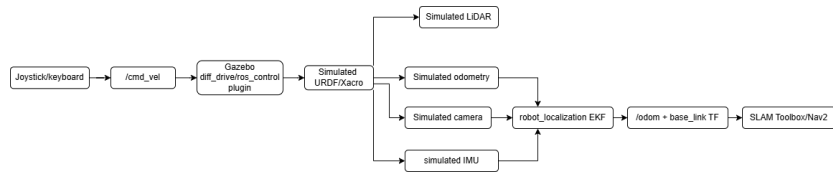


Figure 3: Software control pipeline overview

In the software based system, movement commands are generated by a keyboard and are published on the `/cmd_vel` topic. Instead of being sent to a

physical robot, these commands are processed by a Gazebo `diff_drive` or `ros2_control` plugin, which moves the simulated robot model defined by the URDF/Xacro files. As the robot moves within the virtual environment, Gazebo generates simulated sensor data, including odometry, IMU, LiDAR, and camera information. The odometry and IMU data are used by the `robot_localization` Extended Kalman Filter (EKF) to produce accurate robot pose estimates and the `odom` to `base_link` transformation. The simulated LiDAR data is used by SLAM Toolbox and Nav2 for mapping, localization, and path planning, while the simulated camera data can be utilized for computer vision tasks such as object detection and obstacle avoidance. This simulation-based architecture enables the development, testing, and validation of robotic algorithms without requiring physical hardware.

3 Task 1: Simulation based teleoperation

For this first step, the pipeline contains only:

- M3Pro base model
- Gazebo physics
- robot state publisher
- Keyboard or joystick teleoperation
- `cmd vel`
- Gazebo movement plugin

3.1 keyboard teleoperation

Previous hardware pipeline:

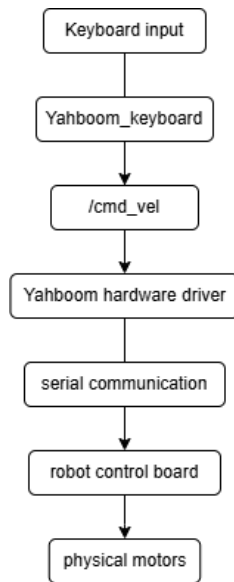


Figure 4: Hardware pipeline overview

The existing `yahboomcar_keyboard.py` node can be used without any changes.

New software pipeline:

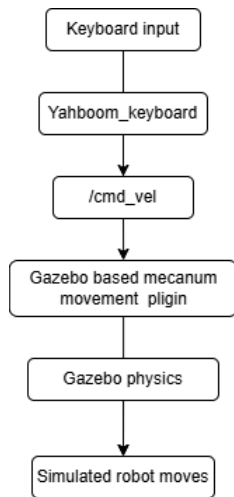


Figure 5: Software pipeline overview

4 understand the repo structure and debug the errors

What's actually in this repo

- Two workspaces: yahboomcar_ws (arm driver, robot description/URDF, MoveIt2 config, laser, depth camera) and M3Pro_ws (navigation2, SLAM, calibration, and a big “largemodel” AI/voice stack).
- The Docker_M3Pro-*.sh scripts pull an image from 192.168.2.51:5000 — that's a private LAN registry inside Yahboom's factory network, not reachable from your our system at all.
- Hardware drivers are hardcoded, e.g. arm_driver.py does Rosmaster("/dev/ttyUSB0") using a proprietary Rosmaster_Lib pip package. Without hardware, this will throw `SerialException: could not open port /dev/ttyUSB0`.
- The only working "no-hardware" launch files are plain URDF viewers (display.launch.py), which just start robot_state_publisher + joint_state_publisher.

4.1 RViz2 / Robot Description Debugging

During comiling the repo, I encountered some errors related to missing packages and dependencies. To resolve these issues, I followed these steps:

```
[robot_state_publisher-2] [INFO] [1783430310.599880042] [robot_state_publisher]:
got segment rwheel1
[robot_state_publisher-2] [INFO] [1783430310.599882271] [robot_state_publisher]:
got segment rwheel2
[joint_state_publisher-1] [INFO] [1783430310.789573185] [joint_state_publisher]:
Waiting for robot_description to be published on the robot_description topic...
^C[WARNING] [launch]: user interrupted with ctrl-c (SIGINT)
[robot_state_publisher-2] [INFO] [1783430729.461275397] [rclcpp]: signal_handler
(SIGINT/SIGTERM)
[INFO] [robot_state_publisher-2]: process has finished cleanly [pid 29831]
[INFO] [joint_state_publisher-1]: process has finished cleanly [pid 29829]
pawan@pawan-LQ-15IRX10: ~/yahboomcar_ws_test/yahboomcar_ws$ grep -rn "dofbot_M3P
ro_description" src/
src/yahboom_M3Pro_description/urdf/M3Pro.urdf:1098:         filename="package:/
/dofbot_M3Pro_description/meshes/arm_base_Link.STL" />
src/yahboom_M3Pro_description/urdf/M3Pro.urdf:1113:         filename="package:/
/dofbot_M3Pro_description/meshes/arm_base_Link.STL" />
src/yahboom_M3Pro_description/urdf/.ipynb_checkpoints/M3Pro-checkpoint.urdf:1098
:         filename="package://dofbot_M3Pro_description/meshes/arm_base_Link.STL"
/>
src/yahboom_M3Pro_description/urdf/.ipynb_checkpoints/M3Pro-checkpoint.urdf:1113
:         filename="package://dofbot_M3Pro_description/meshes/arm_base_Link.STL"
/>
pawan@pawan-LQ-15IRX10: ~/yahboomcar_ws_test/yahboomcar_ws$
```

Figure 6: URDF errors

The robot model initially showed an error because the package name in the URDF file was incorrect. The error was found at lines 1098 and 1113. According to the correction, rebuilt the workspace using `colcon build`, and launched the model again. After the correction, the robot appeared successfully without errors.

4.2 RViz2 / Controlling the Robot movement

I tested the Chassis Control Course, especially section 3.3.1 - Controlling the Car's Speed, without connecting the real robot hardware.

After launching the robot description using `display.launch.py`. The URDF model loaded correctly in RViz, and the robot was visible without errors. I then checked the active ROS 2 nodes and topics. Only visualization-related nodes such as robot state publisher, joint state publisher, and RViz were running. There was no chassis driver or motor control node.

```
ns. It should start with a lower case letter and only contain lower case letters
, digits, underscores, and dashes.
Starting >>> yahboom_M3Pro_description
Finished <<< yahboom_M3Pro_description [0.61s]

Summary: 1 package finished [0.79s]
pawan@pawan-LQ-15IRX10:~/yahboomcar_ws_test/yahboomcar_ws$ source install/setup
.bash
pawan@pawan-LQ-15IRX10:~/yahboomcar_ws_test/yahboomcar_ws$ ros2 node list
/joint_state_publisher
/robot_state_publisher
/rviz
/transform_listener_impl_62f43ba51730
pawan@pawan-LQ-15IRX10:~/yahboomcar_ws_test/yahboomcar_ws$ ros2 topic list
/clicked_point
/goal_pose
/initialpose
/joint_states
/parameter_events
/robot_description
/rosout
/tf
/tf_static
pawan@pawan-LQ-15IRX10:~/yahboomcar_ws_test/yahboomcar_ws$
```

Figure 7: Node and Topic List

Next, a velocity command was published to the `/cmd_vel` topic:

```
ros2 topic pub /cmd_vel geometry_msgs/msg/Twist \
"{linear: [x: 0.1, y: 0.0, z: 0.0], angular: [x: 0.0, y: 0.0, z: 0.0]}" --once
```

The command continuously showed:

```
Waiting for at least 1 matching subscription(s)...
```

This happened because no node was subscribed to `/cmd_vel`. On the real robot, the chassis driver receives this command, sends motor speed values to the STM32 controller through serial communication, reads encoder data, and publishes odometry. Since the hardware and chassis driver were not available, the velocity command had no receiver.

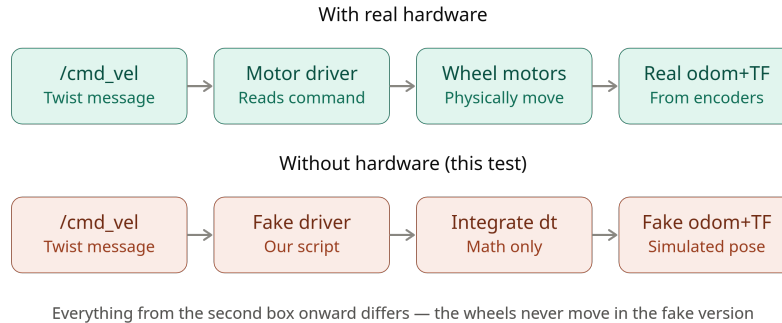


Figure 8: Real vs. Fake Control Pipeline

To test the chassis-control command without the real robot hardware, a temporary node called `fake_odom_driver.py` was added. The original command publishes velocity values on `/cmd_vel`, but no real motor driver is available to receive them. Therefore, the robot cannot move in RViz using only `robot_state_publisher`.

The fake driver works as a simple software replacement for the hardware driver. It subscribes to `/cmd_vel` and reads the commanded forward, sideways, and rotational velocities. These values are stored as:

```

linear.x -> forward/backward movement
linear.y -> sideways movement for mecanum wheels
angular.z -> rotation

```

The node then calculates the robot’s new position over time using simple dead-reckoning. It updates the robot’s x , y , and heading angle based on the latest velocity command. After calculating the pose, it publishes an odometry message on `/odom` and broadcasts the TF transformation:

```
odom -> base_link
```

RViz uses this TF transformation to update the position of the complete robot model. The robot links and joints are still displayed by `robot_state_publisher`, while the fake driver changes the position of the robot base.

4.3 Set Gazebo Environment / Control the Robot movement

“latex

4.4 Base Movement Control in Gazebo Simulation

4.4.1 Objective

The objective of this stage was to make the ROSMASTER M3Pro mobile base movable and controllable in Gazebo Classic using a holonomic control method suitable for its mecanum-wheel configuration. The simulated robot should be capable of forward and backward motion, lateral movement, and rotation around its vertical axis.

4.4.2 Control Approach

The mobile base was controlled using the Gazebo Classic `libgazebo_ros_planar_move.so` plugin. This plugin subscribes to the `/cmd_vel` topic and directly applies linear and angular velocities to the robot chassis.

The planar movement plugin supports velocity commands in the longitudinal direction, lateral direction, and yaw rotation. Therefore, the robot can perform holonomic movements such as sideways strafing, which is suitable for a mecanum-wheeled mobile platform.

In this initial simulation approach, the individual wheel-contact kinematics and mecanum roller dynamics are not modelled. Instead, the commanded chassis velocity is applied directly to the robot base. This provides a simple and reliable method for validating keyboard control, navigation commands, odometry, and the overall simulation architecture.

4.4.3 Files Added

The following Xacro files were added to the robot description package:

- `urdf/M3Pro.planar_move.xacro`

This file defines the `M3Pro_gazebo_planar_move` macro. The macro contains the Gazebo planar movement plugin configuration, including the command velocity topic, odometry topic, odometry frame, robot base frame, and odometry publishing settings.

- `urdf/M3Pro.wheel_friction.xacro`

This file defines the `M3Pro_wheel_friction` macro. It configures the Gazebo friction parameters `mu1` and `mu2` for all four wheels. The friction coefficients were reduced to near-zero values because the wheels are not used to physically generate the robot motion when the planar movement plugin is active.

4.4.4 Files Modified

The following existing files were modified:

- `urdf/M3Pro.urdf.xacro`

The planar movement and wheel-friction Xacro files were included in the main robot description. Their corresponding macros were then called so that the plugin and friction parameters became part of the robot model loaded into Gazebo.

- `launch/gazebo_display.launch.py`

A `joint_state_publisher` node was added to publish joint states for non-driven joints, including the wheel and camera joints. This allowed RViz2 to resolve and display the complete TF tree of the robot.

4.4.5 Implementation and Testing

After integrating the planar movement plugin, the robot was tested by publishing velocity commands directly to the `/cmd_vel` topic. The following types of movement were successfully verified:

- Forward and backward movement using the linear velocity component `linear.x`.
- Sideways strafing using the lateral velocity component `linear.y`.
- Clockwise and counter-clockwise rotation using the angular velocity component `angular.z`.

An example command used to test forward movement is shown below:

```
ros2 topic pub /cmd_vel geometry_msgs/msg/Twist \
"{linear: {x: 0.3, y: 0.0, z: 0.0},
 angular: {x: 0.0, y: 0.0, z: 0.0}}"
```

Similarly, lateral movement was tested by assigning a non-zero value to `linear.y`, while rotational movement was tested by assigning a non-zero value to `angular.z`.

4.4.6 Wheel Joint Type and Limit Configuration

The M3Pro drive wheels must be able to rotate continuously during forward, backward, lateral, and rotational motion. Therefore, the wheel joints should be defined as `continuous` joints rather than `revolute` joints.

A `revolute` joint requires lower and upper angular limits. For example:

```
<joint name="wheel_joint" type="revolute">
  <limit lower="-1.0"
        upper="1.0"
        effort="100"
        velocity="100"/>
</joint>
```

In this configuration, the wheel can rotate only between -1 radian and $+1$ radian. This corresponds to a total angular range of:

$$2 \text{ rad} \approx 114.6^\circ$$

Therefore, the wheel can rotate only approximately one-third of a complete revolution before reaching its position limit. Once the joint reaches either the lower or upper limit, Gazebo prevents any further rotation. Although the velocity limit is set to 100 rad/s , this does not remove the angular position restriction.

For this reason, a **revolute** joint is more suitable for limited-motion joints such as robot arm joints, steering joints, gripper joints, or camera tilt joints.

For the M3Pro drive wheels, the correct configuration is:

```
<joint name="wheel_joint" type="continuous">
  <limit effort="10"
        velocity="10"/>
</joint>
```

A **continuous** joint can rotate indefinitely in both directions. Therefore, the **lower** and **upper** position limits are not required.

The **effort** parameter defines the maximum torque that may be applied to the joint, while the **velocity** parameter defines the maximum permitted angular velocity. In this simulation, setting both parameters to non-zero values allowed the wheel joints to move freely when the planar movement plugin commanded the robot chassis.

The previous wheel joint configuration used:

```
<limit lower="0"
        upper="0"
        effort="0"
        velocity="0"/>
```

This configuration did not permit useful wheel motion. The zero velocity limit prevented wheel rotation, while the zero effort limit did not allow torque to be applied to the joint. In addition, the identical lower and upper limits represented a fixed angular position.

After changing the wheel joints to **continuous** joints and using:

```
<limit effort="10"
        velocity="10"/>
```

the wheels were able to rotate without angular restrictions. This eliminated the abnormal wheel resistance observed during chassis movement and helped resolve the Gazebo tipping behaviour.

4.4.7 Result

The M3Pro mobile base is fully functional in the Gazebo Classic simulation. Forward movement, backward movement, lateral strafing, and rotation were successfully tested. The wheel-friction modification eliminated the chassis-tipping problem, while the planar movement plugin provided stable holonomic motion.

The simulated odometry and TF data were also verified using ROS 2 command-line tools and RViz2. Therefore, the base movement stage provides a functional foundation for the subsequent integration of keyboard teleoperation, SLAM, localization, obstacle avoidance, and Nav2-based autonomous navigation.

4.5 Lidar Scan Merging and SLAM Mapping

4.5.1 Objective

The objective of this task was to combine the data from the two lidar sensors of the M3Pro robot into a single unified laser scan and use the merged scan for Simultaneous Localization and Mapping (SLAM). The final goal was to generate and save an occupancy-grid map of the simulated test environment.

4.5.2 Background

The M3Pro robot is equipped with two 360-degree lidar sensors. Their corresponding coordinate frames are defined as `laser0_frame` and `laser1_frame`. The sensors are mounted at diagonally opposite corners of the robot chassis.

Although each lidar provides a complete rotational scan, parts of its field of view are blocked by the robot body. This creates self-occlusion and causes incomplete measurements around the chassis. By combining both lidar scans, the blind region of one sensor can be covered by the second sensor, producing a more complete representation of the surrounding environment.

4.5.3 Lidar Scan Merging Approach

The existing `ira_laser_tools` package was used to merge the two lidar scans. This package was already available in the `M3Pro_ws` workspace and provides the `laserscan_multi_merger` node.

The merger node subscribes to the two raw lidar topics:

- `/scan0`
- `/scan1`

The measurements from both sensors are transformed into a common reference frame using the available TF transformations. The combined scan is then published on:

`/scan_multi`

This merged topic provides a more complete laser scan around the robot and is used as the input for the SLAM system.

4.5.4 SLAM Approach

The `slam_toolbox` package was used to generate the occupancy-grid map. The existing `slam_mapping` package was configured to run `slam_toolbox` in online asynchronous mode.

In online asynchronous mode, the SLAM node continuously processes incoming laser scans while the robot moves through the environment. It estimates the robot pose using scan matching and incrementally updates the occupancy-grid map.

The merged lidar topic `/scan_multi` was used as the primary sensor input instead of using either lidar individually.

4.5.5 Configuration Changes

The following configuration files were reviewed and modified.

`ira_laser_tools/config/laserscan_merge.yaml` The input scan topic names were verified to match the lidar topics published by the Gazebo sensor plugins:

```
scan0: /scan0
scan1: /scan1
```

The merged laser scan was configured to be published on:

```
/scan_multi
```

`slam_mapping/config/mapper_params_online_async.yaml` The following parameters were updated:

- The `base_frame` parameter was changed from `base_footprint` to `base_link`, because the current robot URDF does not contain a `base_footprint` frame.
- The `scan_topic` parameter was changed from `/scan` to `/scan_multi`, allowing `slam_toolbox` to use the merged lidar data.
- The `max_laser_range` parameter was increased from 6.0 metres to 8.0 metres to match the effective range of the simulated lidar sensors and avoid unnecessarily clipping valid measurements.

The relevant configuration is represented as follows:

```
base_frame: base_link
scan_topic: /scan_multi
max_laser_range: 8.0
```

4.5.6 Implementation and Testing Process

The complete lidar merging and mapping pipeline was tested step by step.

1. Both raw lidar topics were first verified individually:

```
ros2 topic echo /scan0
ros2 topic echo /scan1
```

This confirmed that both Gazebo lidar plugins were publishing valid `sensor_msgs/msg/LaserScan` messages.

2. The `ira_laser_tools` workspace was built, and the `laserscan_multi_merger` node was launched.
3. The merged topic was verified using:

```
ros2 topic echo /scan_multi
```

The topic publishing frequency was also checked using:

```
ros2 topic hz /scan_multi
```

4. The raw and merged scans were visualized in RViz2. The merged scan successfully covered regions that were blocked in the individual lidar scans, reducing the blind areas caused by the robot chassis.
5. The `slam_mapping` package was built and the following launch file was used:

```
ros2 launch slam_mapping online_async_launch.py
```

This launch file starts the main SLAM node without the additional IMU, EKF, and laser-filter processing chain used in the real-hardware launch configuration.

6. RViz2 was opened using the existing visualization launch file:

```
ros2 launch slam_mapping slam_view.launch.py
```

The LaserScan display topic in RViz2 was changed to `/scan_multi`.

7. The robot was manually driven through the simulated maze using the existing teleoperation node, which publishes velocity commands on the `/cmd_vel` topic.
8. As the robot moved, `slam_toolbox` processed the merged laser scans and progressively constructed the occupancy-grid map.

4.5.7 TF Verification

The TF hierarchy was checked to ensure that the mapping system received the correct robot pose information. The expected transformation chain was:

`map → odom → base_link`

The `odom` to `base_link` transformation was published by the Gazebo planar movement plugin and represented the local motion of the simulated robot.

The `map` to `odom` transformation was published by `slam_toolbox`. This transformation corrected accumulated odometry errors based on lidar scan matching and aligned the robot trajectory with the generated map.

The complete TF tree was verified using RViz2 and ROS 2 TF tools.

4.5.8 Map Generation and Saving

After the robot had explored the required area and the maze geometry was fully reconstructed, the generated occupancy-grid map was saved using the existing map-saving launch file:

```
ros2 launch slam_mapping save_map.launch.py
```

The map-saving process used `nav2_map_server` and generated the following files:

- `yahboom_map.yaml`
- `yahboom_map.pgm`

The `PGM` file contains the occupancy-grid image, while the `YAML` file contains the associated map metadata, including resolution, origin, occupancy threshold, free-space threshold, and map image location.

4.5.9 Result

The complete dual-lidar SLAM pipeline was successfully implemented and tested in Gazebo Classic. Both raw lidar scans were merged into the unified `/scan_multi` topic, and the resulting scan provided improved environmental coverage around the robot.

The merged lidar data was successfully used by `slam_toolbox` to generate a live occupancy-grid map. The reconstructed maze geometry closely matched the ground-truth Gazebo world, confirming that the lidar transformations, scan merging, odometry, TF hierarchy, and SLAM configuration were functioning correctly.

The final map was saved successfully as `yahboom_map.yaml` and `yahboom_map.pgm`.

4.5.10 Next Step

The saved occupancy-grid map will be used in the next stage of the project for Nav2-based localization, global path planning, local path planning, and autonomous navigation. The initial launch and configuration structure for this stage is already available in the `M3Pro_navigation` package.